

CAMPUS MARKET

Online Marketplace Platform

System Documentation

Design logic, page-by-page walkthrough and source code notes

Prepared by GROUP A

Kampala University Masaka

E-commerce Project, 2026

Contents

1.	Introduction	1
2.	Who uses the site: the three roles	1
3.	How the whole system fits together	2
4.	Folder and file layout	3
5.	The data layer: how information is stored	4
6.	The public site: search, product cards, cart, checkout	6
7.	The business dashboard	8
8.	The admin panel	11
9.	The chat assistant	13
10.	Payments: MTN MoMo and Mastercard	14
11.	The free trial for the first 100 businesses	15
12.	Known limitations and what a live version still needs	15
13.	Version 2 updates: Business AI, cloud-sync fix, payments and messaging	17
14.	Closing notes	19

Page numbers refer to the printed layout of this document and may shift slightly depending on the viewer used to read it.

1. Introduction

CAMPUS MARKET is an online marketplace built for Group A's e-commerce project at Kampala University Masaka. The idea behind it is simple: give small and medium businesses around Masaka, Ssembabule, Kampala, Gayaza and Kyotera a single place to list what they sell, and give shoppers one place to search across all of them, compare prices, and pay without needing to visit each business in person.

This document is written for whoever needs to understand how the site works without reading every line of code first — a supervisor marking the project, a new team member picking up the code, or the group itself six months from now trying to remember why a function was written a certain way. It covers the reasoning behind the structure of the site, walks through what happens on each page, and quotes the actual code where it helps to see it rather than just describe it.

The current build is a working front-end prototype: everything a user, a business or the admin does on the site actually works in the browser, but the "database" is the browser's own local storage rather than a server. Section 12 explains exactly what that means and what would need to change to run this for real paying customers.

2. Who uses the site: the three roles

The whole design is organised around three kinds of people, and almost every decision in the code comes back to keeping these three separate.

2.1 The user (shopper)

Anyone browsing the site can look at products without an account. To buy something, add it to a cart, or message a business, they need to sign up with an email address and a password. Once signed in, a user stays on the public site — they never see a dashboard, because there is nothing for a shopper to manage beyond their own cart and orders.

2.2 The business

A business signs up with a business name instead of a personal name, picks a category (electronics, fashion, office, machinery, home and living, or agriculture) and a home location. Logging in with a business account sends the person straight to `business-dashboard.html` instead of the public homepage. From there they can upload products with photos, prices and discounts, watch their sales, and see the same product on the public site the moment they save it.

2.3 The admin

There is exactly one admin login, matching the brief: the name **ADMIN GROUP A** and the password **KU MASAKA**. This is checked directly against a constant in the code (`js/data.js`) rather than stored as a regular account, because the brief describes a single fixed administrator rather than a role many people can sign up for. The admin can see every user and business, remove listings or accounts that break the rules, watch traffic and growth, read feedback, and set the mobile money and card accounts that businesses eventually pay their subscription into.

3. How the whole system fits together

The site is built as three HTML pages that share one styling file and one data file. There is no separate back-end server in this version — the browser does the work of a server by keeping all the information in a storage area built into it, called `localStorage`. The diagram below is a rough sketch of how a click travels through the code.

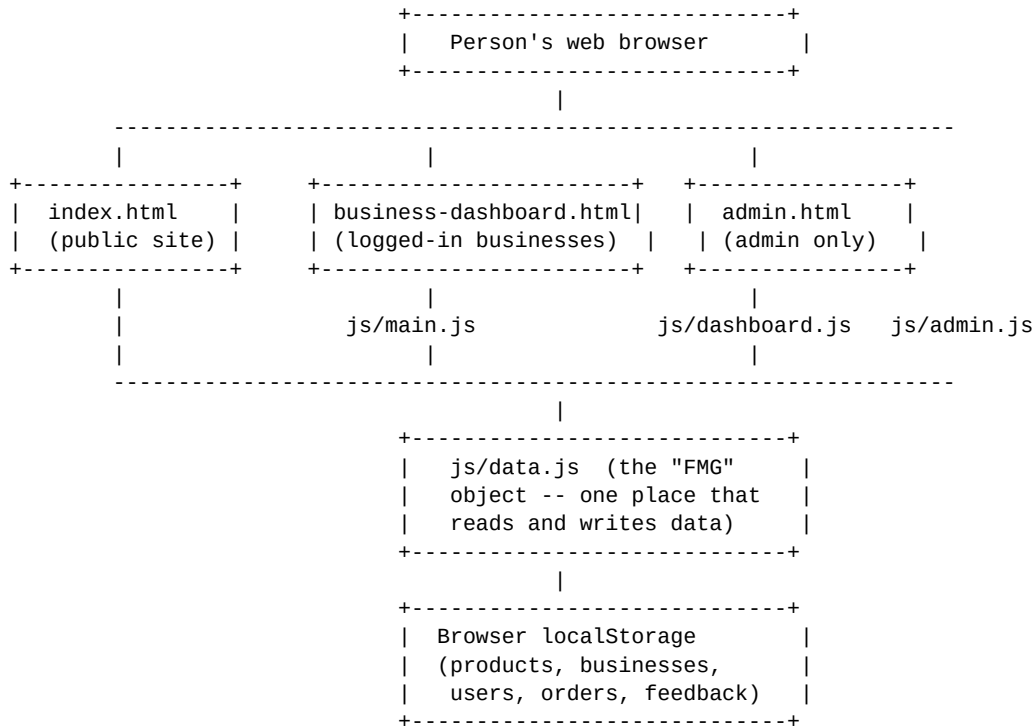


Figure 1. Every page loads `data.js` first, and only `data.js` is allowed to talk to `localStorage` directly. The other files ask `data.js` for what they need through the FMG object.

This matters for one practical reason: if the group later builds a real server, the only file that needs to change is `data.js`. Every button, form and chart in the other files calls functions like `FMG.getProducts()` or `FMG.saveProducts(...)` — none of them know or care whether that data came from `localStorage` or from a network request. Section 5 goes into this in more detail.

3.1 Why three separate HTML files instead of one page

An early option was to build one single page and hide/show the admin panel or dashboard with JavaScript. We chose three separate files instead, because a business dashboard and an admin panel are private areas that should not even be downloaded into a stranger's browser tab unless they are meant to be there, and because it mirrors how the finished, server-backed version would work: different URLs for the public shop, the business area, and the admin area.

3.2 Logic flow chart of the website

Figure 2 lays out the same journey in flowchart form: what a visitor sees the moment the site opens, the point where the site asks them to sign up or log in, and the three separate paths that follow — shopper, business, and admin — each ending in the outcome that role is there to reach. It is meant to be read top to bottom: the diamond shapes are the decision points in the code, the rounded boxes are steps the person actually takes, and the pill-shaped boxes are where a particular journey finishes.

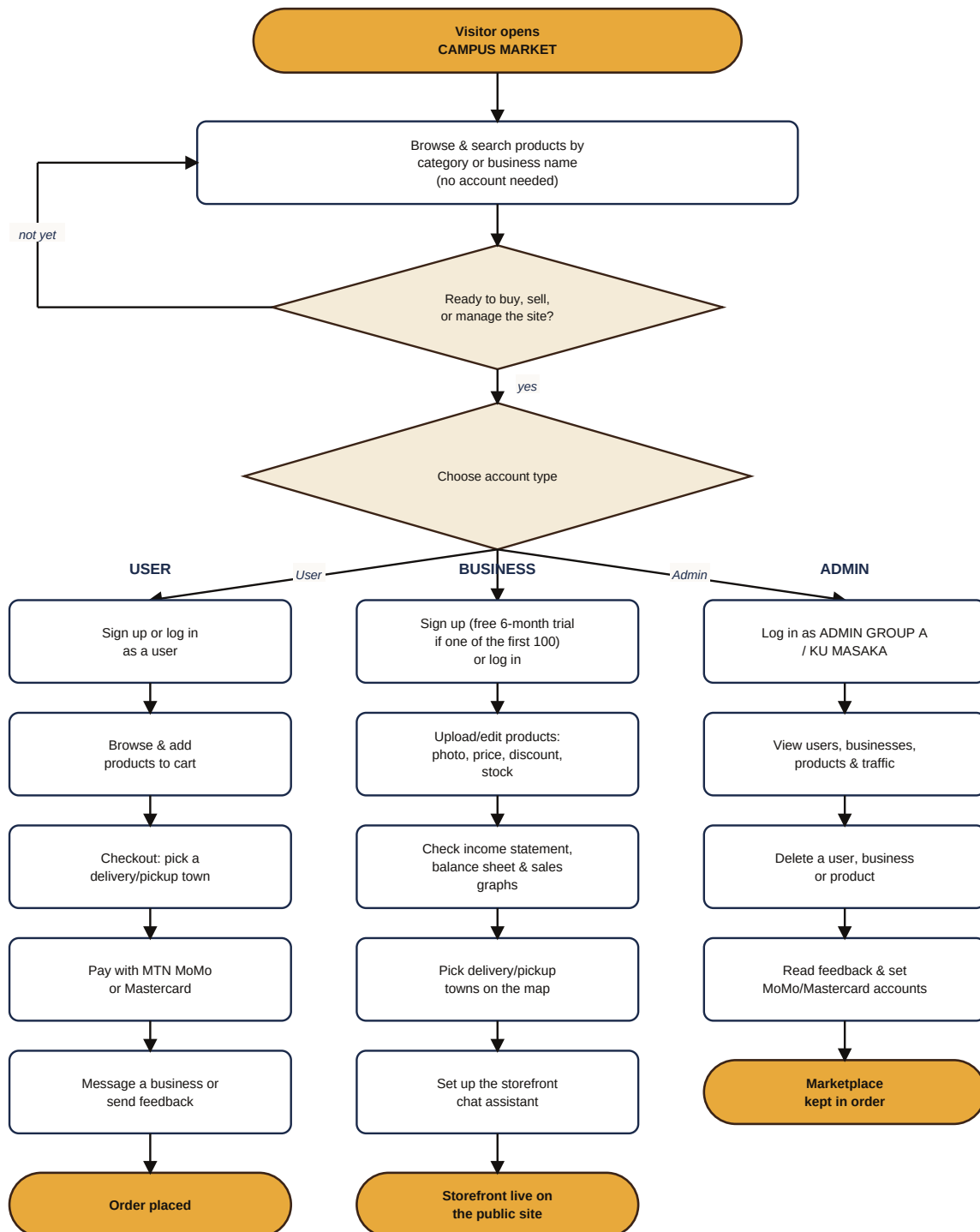


Figure 2. Logic flow of CAMPUS MARKET, from first visit to the outcome of each of the three roles described in section 2. "Not yet" loops a visitor back to browsing instead of forcing them to sign up before they've found something they want.

4. Folder and file layout

This is the folder that should be pushed to the repository, replacing the previous Swift Market files:

```
fmg/
|-- index.html           Public marketplace (home page)
|-- business-dashboard.html Business account area
|-- admin.html          Admin login + admin panel
|-- README.md           Setup notes and what to change before going live
|-- css/
|   |-- style.css        All colours, fonts and layout rules
|-- js/
|   |-- data.js           Seed data + the FMG data-access object
|   |-- main.js           Logic for index.html
|   |-- dashboard.js      Logic for business-dashboard.html
|   |-- admin.js          Logic for admin.html
```

Keeping style in one CSS file and behaviour in one JavaScript file per page (plus the shared data file) means a change to, say, the colour of a button only has to happen once, and a bug in the checkout flow can only be in one file: main.js.

5. The data layer: how information is stored

Every product, business, user, order and message is kept as plain JavaScript objects, saved into localStorage as JSON text. The very first time the site loads, a function called fmgSeed() runs once and fills the storage with a handful of example businesses and products (see js/data.js) so the site is not empty on first look. After that first run, a flag called fmg_seeded stops it from ever overwriting real data again.

5.1 The FMG object

All reading and writing of data happens through one object, so that nothing else in the codebase needs to know the word 'localStorage'. This is what it looks like, shortened to the important part:

```
const FMG = {
  getProducts() { return fmgLoad("fmg_products", []); },
  saveProducts(list) { fmgSave("fmg_products", list); },

  getBusinesses() { return fmgLoad("fmg_businesses", []); },
  saveBusinesses(list) { fmgSave("fmg_businesses", list); },

  getUsers() { return fmgLoad("fmg_users", []); },
  saveUsers(list) { fmgSave("fmg_users", list); },

  getSession() { return fmgLoad("fmg_session", null); },
  setSession(v) { fmgSave("fmg_session", v); },

  isFreeTrialAvailable() {
    return fmgLoad("fmg_registered_business_count", 0) < FMG_FREE_TRIAL_LIMIT;
  }
  // ...and so on for orders, cart, feedback, traffic and payment accounts
};
```

fmgLoad and fmgSave are two small helper functions right above this object. fmgLoad reads a key from localStorage and turns the saved text back into a JavaScript object; fmgSave does the reverse. Wrapping both in a try/catch means that if storage is ever full or blocked (for example, in a browser's private mode), the site keeps working instead of crashing.

5.2 Product photos without slowing the site down

One requirement from the brief was that uploaded pictures should not make the site slow. Photos are never stored at their original size. When a business selects a photo, handleImageSelect() in dashboard.js draws it onto a hidden canvas element, shrinks it so its longest side is at most 700 pixels, and re-saves it as a JPEG at roughly 72% quality before it is ever written to storage:

```
function handleImageSelect(input) {
  const file = input.files[0];
  const reader = new FileReader();
  reader.onload = (e) => {
    const img = new Image();
    img.onload = () => {
      const maxDim = 700;
      let { width, height } = img;
      if (width > height && width > maxDim) { height *= maxDim / width; width = maxDim; }
      else if (height > maxDim) { width *= maxDim / height; height = maxDim; }
      const canvas = document.createElement("canvas");
      canvas.width = width; canvas.height = height;
      canvas.getContext("2d").drawImage(img, 0, 0, width, height);
      pendingImageDataUrl = canvas.toDataURL("image/jpeg", 0.72);
    };
    img.src = e.target.result;
  };
  reader.readAsDataURL(file);
}
```

On top of that, every product image on the public grid carries `loading="lazy"`, so the browser only downloads a photo once it is about to scroll into view, rather than loading every product photo on the page at once.

5.3 What is stored, in plain terms

Storage key	What it holds
fmg_products	Every product: name, category, price, discount, stock, description, photo, owning business
fmg_businesses	Every registered business, including their free-trial dates
fmg_users	Every registered shopper
fmg_orders	Every completed (demo) checkout
fmg_cart	The current shopper's cart
fmg_feedback	Messages sent to a business, or to the admin, or general site feedback
fmg_traffic	A day-by-day visit count used for the admin's traffic chart
fmg_payment_accounts	The MoMo number and Mastercard merchant ID the admin has set
fmg_session	Who is currently logged in, and as what role

6. The public site: search, product cards, cart, checkout

index.html is the shop window. Its job is to load the shared header (search bar and category strip), show a grid of products, and let a signed-in shopper build a cart and pay.

6.1 Category and search filtering

Both the category strip under the header and the search box read and write the page's own URL. Choosing "Fashion" simply reloads the page as `index.html?cat=fashion`; typing a search term and pressing Search reloads it as `index.html?q=dress`. This was a deliberate choice: it means a category page or a search result has its own shareable link, and the "current filter" never gets lost if the page is refreshed. `renderProducts()` reads those two URL values and filters the full product list before drawing the grid:

```
const filtered = products.filter(p => {
  const biz = businesses.find(b => b.id === p.bizId);
  const matchesCat = !cat || p.category === cat;
  const haystack = (p.name + " " + p.desc + " " + (biz ? biz.name : "")).toLowerCase();
  const matchesQ = !q || haystack.includes(q);
  return matchesCat && matchesQ;
});
```

Notice the search also looks inside the business's name, not just the product name — the brief asked for search "according to categories of the product or business names", so typing a business's name finds everything that business sells.

6.2 Adding to cart and the sign-in wall

A shopper cannot add to cart, checkout, or message a business while signed out. Every "Add to cart" button calls `addToCartGuarded()` rather than `addToCart()` directly, and that guard is what checks whether someone is actually signed in as a user before letting the real function run:

```
function addToCartGuarded(productId) {
  const session = FMG.getSession();
  if (!session || session.type !== "user") {
    openAuthModal("login");
    return;
  }
  addToCart(productId);
}
```

If nobody is signed in, this simply opens the login/sign-up window instead of silently failing, so the shopper understands why nothing happened.

6.3 Notifying the business when an item is added to cart

The brief specifically asks for the business to be notified when a shopper adds one of their items to their cart. Rather than sending a real message (there is no server to send it from), this version writes a small notification record that the business dashboard reads and displays under "Recent activity":

```
function notifyBusinessOfCartAdd(productId) {
  const p = FMG.getProducts().find(x => x.id === productId);
  const notes = fmgLoad("fmg_biz_notifications", []);
  notes.unshift({ id: FMG.uid("note"), bizId: p.bizId,
    message: `A shopper added "${p.name}" to their cart.`, at: new Date().toISOString() });
  fmgSave("fmg_biz_notifications", notes.slice(0, 50));
}
```

6.4 Checkout and payment

The cart drawer keeps a running total; pressing "Checkout" opens a modal asking for a delivery or pickup point (one of the six towns) and a payment method: MTN MoMo or Mastercard. Choosing MoMo asks for a phone number; choosing Mastercard asks for card details. Submitting the form calls `submitPayment()`, which creates an order record, reduces the stock of everything that was bought, empties the cart, and marks the order "paid (demo)". Section 10 explains exactly why this is a demo and what real payment collection would need.

7. The business dashboard

business-dashboard.html only works for someone signed in as a business. The very first thing dashboard.js does on page load is check this and send anyone else back to the homepage:

```
function guardBusinessSession() {
  const session = FMG.getSession();
  if (!session || session.type !== "business") {
    window.location.href = "index.html";
    return null;
  }
  const biz = FMG.businessById(session.id);
  if (!biz) { FMG.clearSession(); window.location.href = "index.html"; return null; }
  return biz;
}
```

The dashboard is a single page with nine sections, switched by the sidebar without reloading the page (showSection() simply hides every section and un-hides the one that was clicked). Each is described below.

7.1 Overview

Four summary numbers — total revenue, products listed, units sold, and low-stock items — plus a short activity feed built from the cart notifications described in section 6.3.

7.2 Products

A table of everything the business has listed, each row offering Edit or Delete. "Upload product" opens a form with a photo picker (using the compression code from section 5.2), category, price, discount percentage, stock count and a description. Saving a new product calls FMG.saveProducts() with the new item appended — and because index.html reads from the same storage, the product appears on the public site immediately.

7.3 Sales and orders

Every completed checkout that included one of this business's products is listed here with date, product, quantity, delivery point and revenue. This list is built by scanning every order in storage and keeping only the line items that belong to this business's product IDs (see bizOrderLines() in dashboard.js).

7.4 Income statement and balance sheet

These are simplified, clearly-labelled estimates rather than full accounting: revenue is the real total from orders, cost of goods sold is estimated at 60% of revenue, and a platform fee is applied only if the business's free trial has ended. The balance sheet treats unsold stock (at its listed price) plus cash from sales as assets. This was a deliberate simplification — the brief asks for these two documents to exist and to update automatically, and building a full ledger with purchase costs, returns and taxes was out of scope for a project of this size, so the assumptions are written directly next to the numbers on the page.

7.5 Progression graph and sales per month

Both are drawn with Chart.js, a small charting library loaded from a public CDN link in business-dashboard.html. monthlyRevenueSeries() groups the last six months of order data by month; if a business is brand new and has no orders yet, a light demonstration trend is filled in so the chart is not empty, clearly generated rather than pretending to be real sales.

7.6 Delivery and pickup locations

The six towns from the brief — Masaka, Ssembabule, Kampala, Gayaza, Kyotera, and Kampala University Masaka — are hard-coded with approximate map coordinates in js/data.js (FMG_LOCATIONS). A business ticks which of these it can deliver to or offer pickup from; the choice is saved per business and shown on an actual map using Leaflet, an open-source mapping library, with OpenStreetMap as the underlying map data.

No Google Maps API key is needed, which avoids a billing setup for a student project:

```
bizMap = L.map("map").setView([0.0, 31.9], 8);
L.tileLayer("https://{s}.tile.openstreetmap.org/{z}/{x}/{y}.png", {
  attribution: "&copy; OpenStreetMap contributors", maxZoom: 17
}).addTo(bizMap);
FMG.locations.forEach(loc => {
  const active = selectedLocations.includes(loc.id);
  L.circleMarker([loc.lat, loc.lng], { radius: active ? 9 : 6,
    color: active ? "#E8A93B" : "#1B2A4A" }).addTo(bizMap);
});
```

7.7 Chat assistant settings and messages

A business can choose a tone for their storefront assistant (friendly, formal, or sales-focused) and write their own greeting; this is saved per business so each storefront can feel a little different. The "Messages" section lists any note a shopper sent through "Message seller" on a product page.

8. The admin panel

admin.html shows a plain login form first, not the panel itself. Two fields — admin name and password — are checked directly against the constant defined at the top of data.js:

```
const FMG_ADMIN = { name: "ADMIN GROUP A", password: "KU MASAKA" };

function attemptAdminLogin() {
  const name = document.getElementById("gateName").value.trim();
  const pw = document.getElementById("gatePw").value;
  if (name === FMG_ADMIN.name && pw === FMG_ADMIN.password) {
    FMG.setSession({ type: "admin" });
    checkAdminGate();
  } else {
    document.getElementById("gateError").textContent = "Incorrect admin name or password.";
  }
}
```

This is intentionally simple and, as noted in section 12, is not secure enough for a live public site — an admin password should never be checked in code the browser can read. For a class project demo it satisfies the brief exactly as written.

8.1 What the admin can see and do

- Users — every shopper who has signed up, with a Remove button.
- Businesses — every business, its category, location, and whether it is on the free trial, trial ended, or a paid plan, with a Remove button that also deletes its product listings.
- Products — every product on the entire site, regardless of which business owns it, with a Delete button, matching the brief's "remove an uploaded product" requirement.
- Web traffic — a 30-day line chart. Since there is no real server logging visits in this prototype, the numbers are generated once and stored (fmGenerateTraffic() in data.js) as a stand-in for real analytics.
- Progression graphs — user growth and business growth over the last six months.
- Reports — a button that builds and downloads a plain text summary (totals, trial counts, revenue) using the browser's own file-download mechanism, no server required.
- Feedback — every message sent through "Message seller", the general feedback form, or the footer "Send feedback" link, all in one list.
- Payment accounts — the MoMo number and Mastercard merchant ID that appear to shoppers at checkout and that businesses would pay their subscription fee into.

9. The chat assistant

Both the public site and the business dashboard include a chat assistant. It is a rule-based assistant, not a connection to an external AI service — it looks for keywords in what was typed and returns a matching, pre-written answer. This keeps the feature working with no ongoing cost and no internet dependency, which suited a student project, while still meeting the brief's request for "a chat bot inside the web that will answer to their specific customers":

```
function botReply(text) {
  const t = text.toLowerCase();
  if (t.includes("delivery") || t.includes("pickup")) {
    return "We deliver to or offer pickup at: " + FMG.locations.map(l => l.label).join(", ") + ".";
  }
  if (t.includes("pay") || t.includes("momo") || t.includes("card")) {
    return "You can pay with MTN MoMo or Mastercard at checkout.";
  }
  if (t.includes("discount") || t.includes("offer")) {
    return "Look for the yellow discount badge on a product card.";
  }
  // ...a few more matches for cart, business sign-up, and asking for a human
  return "I can help with delivery points, payments, discounts and your cart. What would you like to know?";
}
```

A note for anyone extending this: a real AI-powered version would call a language model's API from here instead of matching keywords, sending the shopper's message and getting a generated reply back — the rest of the chat window (the message list, the input box, the quick-reply buttons) would not need to change.

10. Payments: MTN MoMo and Mastercard

At checkout, a shopper chooses MTN MoMo or Mastercard and enters the relevant details. Pressing Pay records the order as "paid (demo)" — no money actually moves, and no card or phone number is sent anywhere. This is a placeholder for two reasons: first, taking real payments needs a signed agreement and API access with MTN Mobile Money and a card processor, which is outside what a browser-only prototype can do; second, card numbers should never be handled directly by a website's own JavaScript at all — they need to be collected through a certified payment provider's own secure form, or the site risks failing card industry security rules (PCI DSS) and, more importantly, actually exposing customers' card details.

When the group is ready to accept real payments, the realistic path is: sign up for MTN's Mobile Money Open API (or a local aggregator that already supports it, such as Flutterwave or Pesapal, which also support Visa/Mastercard), move the "Pay" button to call that provider's checkout from a real server, and only mark an order paid once that server receives a confirmed payment webhook — never based on what the browser says.

11. The free trial for the first 100 businesses

The brief asks for the first 100 businesses to register for free, for six months, after which they pay. This is tracked with one running counter, `fmg_registered_business_count`, stored the same way as everything else. Every time a business signs up, `registerBusinessTrialSlot()` checks the counter before incrementing it:

```
isFreeTrialAvailable() {
  return fmgLoad("fmg_registered_business_count", 0) < FMG_FREE_TRIAL_LIMIT;
},
registerBusinessTrialSlot() {
  const n = fmgLoad("fmg_registered_business_count", 0);
  fmgSave("fmg_registered_business_count", n + 1);
}
```

```
    return n < FMG_FREE_TRIAL_LIMIT;  
}
```

A business that registers within the first 100 gets a `trialEndsAt` date six months from their sign-up date, shown as a countdown pill on their dashboard ("Free trial - 42 days left"). Once that date passes, the dashboard's income statement starts applying a platform fee, and the admin's business list shows "Trial ended" instead of "Free trial".

12. Known limitations and what a live version still needs

This build proves out every interaction described in the brief and is genuinely usable for a demonstration, a class presentation, or user testing. Before it could hold real customers' money or personal data, the following would need to change:

- A real server and database. Right now, data lives in one browser's local storage, so a product uploaded on a business's laptop will not appear on a shopper's phone until the two are talking to the same server.
- Real password security. Passwords are currently stored as plain text for demonstration; a live site must hash passwords and never keep the admin password inside code the browser can read.
- Real payment processing, as described in section 10.
- Server-side image storage (for example, cloud storage plus a content delivery network) rather than storing photos as text inside the database, once the number of products grows large.
- HTTPS hosting, and a privacy policy that matches whatever real analytics or data sharing with businesses is eventually switched on, to honour the cookie-consent promise already built into the site.

None of this requires rebuilding the site from scratch. Because every page already goes through the one FMG object described in section 5, connecting a real backend mainly means rewriting the inside of that object to call a server instead of `localStorage` — the HTML, the styling, and almost all of the JavaScript in `main.js`, `dashboard.js` and `admin.js` can stay as they are.

13. Version 2 updates

This section records what changed after the first version of the system was documented. Everything below is already in the delivered code.

13.1 Business AI (many free AI services in one chat)

The earlier rule-based chat window was replaced by a Business AI. It is a router: when a shopper asks a question, the site sends it in the background to a free AI service and shows the answer inside the same chat window, so the visitor never leaves CAMPUS MARKET. If a service is busy, out of free quota or down, the router automatically tries the next one. The logic lives in `js/business-ai.js` and the list of services in `js/ai-config.js`.

- **Services:** Google Gemini, Groq, OpenRouter, Mistral, Cerebras, SambaNova, Hugging Face, GitHub Models, NVIDIA NIM and Together AI are listed; any service with an OpenAI-compatible API can be added by copying one line. A service is used only when a free key has been pasted in. Services with no public API (such as the free web versions of ChatGPT or Claude) cannot be included.
- **Choice:** shoppers can leave the selector on Auto or choose a service, and each answer shows which service produced it.
- **Scope:** business and shopping topics only. The live product catalogue is sent with each question so the assistant does not invent items or prices.
- **Pictures:** 'Ask Business AI' on a product page, or the paperclip, sends a photo to a service that can read images.
- **Memory:** every logged-in shopper (and every guest) has a separate saved conversation on their device; the last 14 messages are sent as context, up to 60 are stored, and a Clear button deletes them.
- **Safe fallback:** if no service answers, the window shows built-in help text for delivery, payments and discounts instead of an error.
- **Security:** keys placed in a static site can be read by visitors, so only free-tier keys should be used. For a serious launch, `server/ai-proxy-worker.js` (Cloudflare Workers) keeps the keys on a server.

13.2 Cloud-sync data-loss fix

After Firebase sharing was switched on, the admin panel showed 0 registered businesses and products displayed 'Unknown seller'. The cause was ordering: the site began listening to the shared database immediately, and the first reading of an empty shared collection overwrote a business that had been created on the device before sharing was enabled. The site now checks each shared collection first. If the cloud copy is empty and the device holds data, the device data is uploaded; if the cloud already has data, it is never overwritten by an older local copy. Only after that check does live listening begin. This prevents the problem in future; a business already lost from a live database has to be registered again.

13.3 Payments, delivery fees and messaging

- Each business chooses which of MTN MoMo, Airtel Pay and Mastercard it accepts, and enters its account details in the dashboard.
- At checkout only the methods accepted by every seller in the cart are offered; if there is no overlap the shopper is told why.

- Businesses can set a delivery fee per product and per location, never above UGX 10,000. The cap is enforced in the form and again when saving, and the fee is shown before payment.
- Shoppers can message a seller from a product page; the business replies from its dashboard, and the shopper sees an unread badge. The admin can view conversations read-only.
- Fixes: cart drawer now sits above the header, 'View public site' links work from both dashboards, and new business sign-ups are sent to their dashboard.

13.4 Rebrand

The project name is now CAMPUS MARKET, with a new logo mark, favicon and hero illustration. Internal code names (the FMG object and the fmg_ storage keys) were deliberately left unchanged because visitors never see them.

13.5 What is still simulated

Checkout remains a simulated payment: no real money moves. Income statements and balance sheets are calculated from the orders actually recorded. Collecting real MoMo, Airtel or Mastercard payments needs an account with a licensed payment provider and a small server to verify each payment, and cannot be done safely from a static website alone.

13.6 Testing

Automated browser tests cover navigation links, business sign-up, payment-method saving, the delivery-fee cap, checkout filtering, messaging, the Business AI (with simulated services: automatic fail-over when one is out of quota, manual choice, picture questions routed to image-capable services, memory after reload and the built-in fallback) and the cloud-sync fix (with a simulated shared database). The Business AI services and cloud sync have not yet been run against the real Gemini, Groq and other services or Firebase, so they should be confirmed once on the live site after the key is added.

14. Closing notes

The aim throughout was to build something that actually behaves the way the brief describes, end to end, rather than a set of disconnected mockup screens: a business really can upload a product and watch it appear on the public site within the same demo; a shopper really can add it to a cart, pay, and see stock go down; the admin really can delete that same product and watch it disappear. Anyone marking or extending this project should be able to open the three HTML files, follow the matching JavaScript file, and find the exact behaviour described in this document.

Questions about a specific part of the code are best answered by opening the matching section of this document alongside the file it refers to — each code sample above is copied directly from the project, not simplified or invented for the write-up.

All rights reserved to GROUP A MEMBERS KU MASAKA E-commerce 2026